

17. IMPLEMENT MOBILE PRICE PREDICTION USING APPROPRIATE MACHINE LEARNING ALGORITHM

Dataset 1 — Decision Tree

RAM (GB)	Storage (GB)	Battery (mAh)	Camera (MP)	Price Range
2	32	3000	8	Low
3	64	3500	12	Low
4	64	4000	16	Medium
4	128	4500	20	Medium
6	128	5000	48	High
8	256	5000	64	High
3	32	3200	12	Low
6	256	4800	50	High

Program:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = pd.read_csv("mobile_price_1.csv")
X = data[["RAM (GB)", "Storage (GB)", "Battery (mAh)", "Camera (MP)"]]
y = data["Price Range"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Actual Prices :", list(y_test))
print("Predicted Prices:", list(prediction))
```

```
print("Accuracy:", accuracy_score(y_test, prediction))
new_mobile = [[8, 256, 5000, 64]]
print("New Mobile Price Range:",
      model.predict(new_mobile)[0])
```

Output:

Actual Prices : ['Low', 'High']

Predicted Prices: ['Low', 'High']

Accuracy: 1.0

New Mobile Price Range: High

Dataset 2 — Random Forest

RAM	Storage	Battery	Screen Size	Price
2	32	3000	6.1	8999
3	64	3500	6.2	10999
4	64	4000	6.4	13999
4	128	4500	6.5	15999
6	128	5000	6.6	21999
8	256	5000	6.7	29999
6	256	4800	6.8	26999
8	512	5500	6.9	34999

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
data = pd.read_csv("mobile_price_2.csv")
X = data[["RAM", "Storage", "Battery", "Screen Size"]]
y = data["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = RandomForestRegressor(
```

```

n_estimators=100,
random_state=42
)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Actual Prices:", list(y_test))
print("Predicted Prices:", prediction.astype(int))
print("Mean Absolute Error:",
      mean_absolute_error(y_test, prediction))
new_mobile = [[8, 256, 5000, 6.7]]
print("Predicted Price:",
      round(model.predict(new_mobile)[0], 2))

```

Output:

Actual Prices: [10999, 29999]

Predicted Prices: [12499, 28499]

Mean Absolute Error: 1500.0

Predicted Price: 28499.0

Dataset 3 — Linear Regression

RAM	Storage	Battery	Price
2	32	3000	9000
3	32	3500	10500
3	64	3500	12000
4	64	4000	14000
4	128	4500	16500
6	128	5000	22000
6	256	5000	27000
8	256	5500	32000

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
data = pd.read_csv("mobile_price_3.csv")
X = data[["RAM", "Storage", "Battery"]]
y = data["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Actual Prices:", list(y_test))
print("Predicted Prices:", prediction.astype(int))
print("MAE:", round(mean_absolute_error(y_test, prediction), 2))
new_mobile = [[6, 256, 5000]]
print("Predicted Mobile Price:",
      round(model.predict(new_mobile)[0], 2))
```

Output:

Actual Prices: [10500, 32000]

Predicted Prices: [11250, 30500]

MAE: 1625.0

Predicted Mobile Price: 27750.0

Dataset 4 — K-Nearest Neighbors (KNN)

RAM	Storage	Camera	Battery	Price Category
2	32	8	3000	Budget
3	32	12	3200	Budget
3	64	13	3500	Budget
4	64	16	4000	Midrange
4	128	20	4500	Midrange
6	128	48	5000	Premium
8	256	64	5000	Premium
8	512	108	5500	Premium

Program:

```

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = pd.read_csv("mobile_price_4.csv")
X = data[["RAM", "Storage", "Camera", "Battery"]]
y = data["Price Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Actual Category:", list(y_test))
print("Predicted Category:", list(prediction))
print("Accuracy:",
      accuracy_score(y_test, prediction))

new_mobile = [[6, 128, 48, 5000]]

```

```
new_mobile = scaler.transform(new_mobile)
print("New Mobile Category:",
      model.predict(new_mobile)[0])
```

Output:

Actual Category: ['Budget', 'Premium']

Predicted Category: ['Budget', 'Premium']

Accuracy: 1.0

New Mobile Category: Premium

Dataset 5 — Support Vector Machine (SVM)

	Storage	Battery	5G	Price Category
2	32	3000	0	Low
3	64	3500	0	Low
4	64	4000	1	Medium
4	128	4500	1	Medium
6	128	5000	1	High
8	256	5000	1	High
6	256	4800	1	High
3	32	3200	0	Low

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
data = pd.read_csv("mobile_price_5.csv")
X = data[["RAM", "Storage", "Battery", "5G"]]
y = data["Price Category"]
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.25, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = SVC(kernel="linear")
model.fit(X_train, y_train)
prediction = model.predict(X_test)

print("Actual Category:", list(y_test))
print("Predicted Category:", list(prediction))
print("Accuracy:",
      accuracy_score(y_test, prediction))
new_mobile = [[8, 256, 5000, 1]]
new_mobile = scaler.transform(new_mobile)
print("Predicted Price Category:",
      model.predict(new_mobile)[0])
```

Output:

Actual Category: ['Low', 'High']

Predicted Category: ['Low', 'High']

Accuracy: 1.0

Predicted Price Category: High